คู่มือเล่ม 3: Domain-Driven Design และสถาปัตยกรรมที่สะท้อนโดเมน

การออกแบบซอฟต์แวร์ที่อิงธุรกิจเป็นศูนย์กลาง

> ชุดคู่มือสถาปัตยกรรมซอฟต์แวร์และการบริหารโครงการ
> เล่ม 3 จาก 7 เล่ม
> ฉบับสมบูรณ์ v1.0

คำนำ

Domain-Driven Design (DDD) เป็นแนวทางที่ Eric Evans นำเสนอในปี 2003 และยังคงเป็นเครื่องมือสำคัญที่สุดในการออกแบบซอฟต์แวร์ที่ซับซ้อนจนถึงปัจจุบัน DDD ไม่ใช่ framework หรือ library แต่เป็น **วิธีคิดและชุดเครื่องมือ** ที่ช่วยให้ทีมพัฒนาและผู้เชี่ยวชาญธุรกิจสร้างโมเดลที่สะท้อนธุรกิจจริง

คู่มือเล่มนี้มีเป้าหมายเพื่อให้ผู้อ่าน **เข้าใจหลักการของ DDD ทั้งเชิงกลยุทธ์ (Strategic) และเชิงยุทธวิธี (Tactical)** รวมถึงเชื่อมโยงกับสถาปัตยกรรมรูปแบบอื่น ๆ เช่น Hexagonal, Onion, Clean Architecture และ CQRS เพื่อให้ผู้อ่านสามารถนำไปประยุกต์ใช้ได้จริง

คู่มือเล่มนี้ต่อยอดจากเล่ม 1 (ปฐมบทสถาปัตยกรรม) และเล่ม 2 (Microservices) โดยเฉพาะเรื่องการกำหนดขอบเขตบริการ ซึ่ง DDD เป็นเครื่องมือสำคัญที่สุดในการทำเช่นนั้น

ข้อมูลเอกสาร

รายการ	รายละเอียด
ชื่อเล่ม	Domain-Driven Design และสถาปัตยกรรมที่สะท้อนโดเมน
ชื่อรอง	การออกแบบซอฟต์แวร์ที่อิงธุรกิจเป็นศูนย์กลาง
กลุ่มผู้อ่าน	Software Architect, Tech Lead, Senior Developer, Domain Expert, Business Analyst
ระดับ	กลางถึงสูง
โครงสร้างหลัก	เรื่อง → วัตถุประสงค์ → เป้าหมาย → ข้อดี → ข้อเสีย → ข้อควรระวัง → ตัวอย่าง → การประยุกต์ใช้ → สรุป

รายการ	รายละเอียด
เล่มที่เชื่อมโยง	เล่ม 1 (พื้นฐาน), เล่ม 2 (Microservices), เล่ม 4 (Clean Architecture), เล่ม 5 (ความเสี่ยง), เล่ม 6 (POC), เล่ม 7 (การบริหารโครงการ)
จำนวนบท	18 บท + 7 ภาคผนวก

วิธีใช้เอกสาร

1. **บทที่ 1–3** ใช้สร้างความเข้าใจพื้นฐาน DDD และภาพรวม — อ่านก่อนทุกบท
2. **บทที่ 4–8** เป็น Strategic DDD — ใช้ออกแบบขอบเขตและภาษา
3. **บทที่ 9–14** เป็น Tactical DDD — ใช้ออกแบบโมเดลภายใน bounded context
4. **บทที่ 15–16** เชื่อม DDD กับสถาปัตยกรรมอื่น ๆ — อ่านก่อนออกแบบโครงสร้าง
5. **บทที่ 17–18** เป็นกรณีศึกษาและการประยุกต์ใช้ — อ่านก่อนสรุปแผน
6. **ภาคผนวก** ใช้เป็นเครื่องมือประกอบ workshop และการออกแบบ

โครงสร้างบทมาตรฐาน

ทุกบทในเล่มนี้ใช้โครงสร้างร่วมกันดังนี้

1. เรื่อง
2. วัตถุประสงค์
3. เป้าหมาย
4. หัวข้อหลัก
5. ข้อดีของบท
6. ข้อเสีย/ข้อจำกัด
7. ข้อควรระวัง
8. ตัวอย่าง
9. การประยุกต์ใช้
10. สรุป/ผลลัพธ์
11. เชื่อมโยงกับเล่มอื่น

สารบัญ

ส่วนที่ 1: ภาพรวมและพื้นฐาน DDD

- บทที่ 1 DDD, Hexagonal, Onion, Clean, CQRS: How I Put It All Together
- บทที่ 2 DDD คืออะไร และทำไมต้องใช้
- บทที่ 3 Strategic DDD vs Tactical DDD

ส่วนที่ 2: Strategic DDD (การออกแบบระดับกลยุทธ์)

- บทที่ 4 Domain และ Subdomains
- บทที่ 5 Bounded Contexts
- บทที่ 6 Ubiquitous Language
- บทที่ 7 Context Mapping
- บทที่ 8 การเชื่อม Strategic สู่ Tactical

ส่วนที่ 3: Tactical DDD (การออกแบบระดับยุทธวิธี)

- บทที่ 9 Entities
- บทที่ 10 Value Objects
- บทที่ 11 Aggregates
- บทที่ 12 Repositories
- บทที่ 13 Domain Events
- บทที่ 14 Domain Services และ Factories

ส่วนที่ 4: สถาปัตยกรรมที่สะท้อนโดเมน

- บทที่ 15 โครงสร้างโค้ดที่สะท้อนโดเมน
- บทที่ 16 Reflecting Architecture and Domain in Code
- บทที่ 17 More than Concentric Layers: Hexagonal, Onion, Clean, CQRS

ส่วนที่ 5: การนำไปใช้และกรณีศึกษา

- บทที่ 18 ตัวอย่างการออกแบบระบบด้วย DDD
- บทที่ 19 การประยุกต์ใช้และสรุปเล่ม

ภาคผนวก

- ภาคผนวก A: Domain Storytelling Canvas
- ภาคผนวก B: Bounded Context Canvas
- ภาคผนวก C: Ubiquitous Language Glossary Template
- ภาคผนวก D: Context Map Template
- ภาคผนวก E: Aggregate Design Checklist
- ภาคผนวก F: Domain Event Catalog Template
- ภาคผนวก G: Glossary เฉพาะเล่ม 3

ส่วนที่ 1: ภาพรวมและพื้นฐาน DDD

บทที่ 1: DDD, Hexagonal, Onion, Clean, CQRS: How I Put It All Together

1.1 เรื่อง

ภาพรวมความสัมพันธ์ระหว่าง DDD, Hexagonal Architecture, Onion Architecture, Clean Architecture และ CQRS — แนวคิดที่มักถูกกล่าวถึงร่วมกันแต่มีจุดเน้นต่างกัน

1.2 วัตถุประสงค์

- อธิบายแต่ละแนวคิดและจุดเน้น
- ทำให้เห็นว่าทั้งหมดทำงานร่วมกันได้อย่างไร
- เตรียมพื้นฐานสำหรับบทถัดไป

1.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- อธิบายความแตกต่างของแต่ละแนวคิดได้
- เห็นภาพรวมว่าทั้งหมดประกอบกันอย่างไร
- เลือกใช้ให้เหมาะกับบริบท

1.4 หัวข้อหลัก

1.4.1 ภาพรวมแต่ละแนวคิด

แนวคิด	จุดเน้น	ระดับ
DDD	โมเดลที่สะท้อนธุรกิจ, bounded context, ubiquitous language	Strategic + Tactical
Hexagonal (Ports & Adapters)	แยก core domain ออกจาก infrastructure ผ่าน port/adapter	Architecture
Onion	ชั้น concentric โดย domain อยู่ศูนย์กลาง	Architecture
Clean	แยกชั้นตาม dependency rule, domain อยู่ศูนย์กลาง	Architecture
CQRS	แยก read model กับ write model	Pattern

1.4.2 ความสัมพันธ์

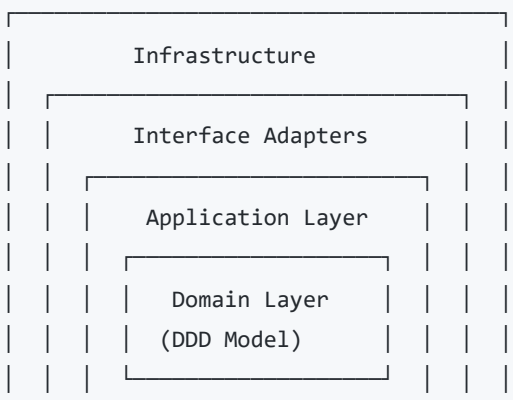
- **DDD** ให้เครื่องมือในการสร้างโมเดลที่สะท้อนธุรกิจ
- **Hexagonal/Onion/Clean** ให้โครงสร้างที่ทำให้โมเดลอยู่ศูนย์กลางและไม่ผูกกับ infrastructure
- **CQRS** เป็น pattern ที่ใช้เสริมเมื่อ read/write มีลักษณะต่างกันมาก

1.4.3 การประกอบกัน

แนวทางที่นิยม:

1. ใช้ **Strategic DDD** กำหนด bounded context
2. ใช้ **Tactical DDD** ออกแบบโมเดลภายใน context
3. ใช้ **Hexagonal/Clean** จัดโครงสร้างโค้ดให้ domain อยู่ศูนย์กลาง
4. ใช้ **CQRS** เมื่อเหมาะสม (read/write ต่างกันมาก)
5. ใช้ **Event Sourcing** เมื่อต้องการ audit trail

1.4.4 ตัวอย่างการประกอบกัน



```
│  │  └──────────────────────┘  │  │
│  └────────────────────────────┘  │
└──────────────────────────────────┘
```

1.5 ข้อดีของบท

- ทำให้เห็นภาพรวมก่อนลงรายละเอียด
- ลดความสับสนระหว่างแนวคิด
- เตรียมพื้นฐานสำหรับบทถัดไป

1.6 ข้อเสีย/ข้อจำกัด

- เป็นภาพรวม ยังไม่ลงรายละเอียด
- แต่ละแนวคิดมีรายละเอียดมาก

1.7 ข้อควรระวัง

- อย่าสับสนระหว่าง DDD กับ Hexagonal/Clean (คนละมิติ)
- อย่าคิดว่าต้องใช้ทั้งหมดพร้อมกัน
- อย่าเลือกตามกระแส

1.8 ตัวอย่าง

ทีมพัฒนาระบบ e-commerce:

- ใช้ **Strategic DDD** แยก bounded context: Sales, Inventory, Shipping
- ใช้ **Tactical DDD** ออกแบบ Aggregate ในแต่ละ context
- ใช้ **Hexagonal** จัดโครงสร้างโค้ดให้ domain ไม่ผูกกับ framework
- ใช้ **CQRS** สำหรับ read model ของ product catalog (query เยอะ)

1.9 การประยุกต์ใช้

- ใช้เป็นแผนที่นำทางก่อนอ่านบทถัดไป
- ใช้สื่อสารกับทีมเรื่องแนวทางที่เลือก
- ใช้ใน architecture review

1.10 สรุป/ผลลัพธ์

ผู้อ่านเห็นภาพรวมของแต่ละแนวคิดและความสัมพันธ์ และพร้อมสำหรับการลงรายละเอียด

1.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 2–3
- ไปสู่เล่ม 4 (Clean Architecture)

- ไปสู่เล่ม 2 (Microservices)

บทที่ 2: DDD คืออะไร และทำไมต้องใช้

2.1 เรื่อง

ความหมาย ที่มา และคุณค่าของ Domain-Driven Design

2.2 วัตถุประสงค์

- อธิบายความหมายของ DDD
- ทำให้เห็นว่าทำไม DDD จึงสำคัญ
- เตรียมพื้นฐานสำหรับบทถัดไป

2.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- อธิบาย DDD ด้วยคำของตนเอง
- เห็นคุณค่าของ DDD ในบริบทที่เหมาะสม
- รู้ว่าเมื่อใดควรใช้ DDD

2.4 หัวข้อหลัก

2.4.1 ความหมายของ DDD

Domain-Driven Design คือแนวทางออกแบบซอฟต์แวร์ที่:

- **ให้ความสำคัญกับโดเมน (domain)** — ธุรกิจที่ระบบบรองรับ
- **ใช้โมเดลที่สะท้อนโดเมน** — ไม่ใช่แค่ data model
- **ทำงานร่วมกับผู้เชี่ยวชาญธุรกิจ** — ไม่ใช่แค่ทีมเทคนิค
- **ใช้ ubiquitous language** — ภาษากลางที่ทุกคนเข้าใจ

2.4.2 ที่มาของ DDD

Eric Evans นำเสนอในหนังสือ *Domain-Driven Design: Tackling Complexity in the Heart of Software* (2003)

แนวคิดหลัก:

- ซอฟต์แวร์ที่ดีต้องสะท้อนโดเมน
- ความซับซ้อนที่แท้จริงอยู่ที่โดเมน ไม่ใช่เทคโนโลยี
- การสื่อสารระหว่างทีมเทคนิคและธุรกิจเป็นหัวใจ

2.4.3 ทำไมต้องใช้ DDD

- **จัดการความซับซ้อน** — ระบบธุรกิจซับซ้อนต้องการโมเดลที่ชัดเจน
- **สื่อสารได้ดีขึ้น** — ubiquitous language ลดการเข้าใจผิด
- **ยืดหยุ่นต่อการเปลี่ยนแปลง** — โมเดลที่สะท้อนธุรกิจปรับตัวได้ง่ายกว่า
- **แยกขอบเขตชัดเจน** — bounded context ลด coupling
- **ทดสอบได้** — domain model ทดสอบได้โดยไม่ต้องพึ่ง infrastructure

2.4.4 เมื่อใดควรใช้ DDD

เหมาะกับ:

- ระบบที่มี business logic ซับซ้อน
- ระบบที่ต้องเปลี่ยนแปลงบ่อยตามธุรกิจ
- ระบบที่มีผู้เชี่ยวชาญธุรกิจพร้อมร่วมมือ
- ระบบขนาดกลาง–ใหญ่

ไม่เหมาะกับ:

- ระบบ CRUD ง่าย ๆ
- ระบบที่ไม่มีความซับซ้อนทางธุรกิจ
- ทีมที่ไม่มี access to domain experts
- โครงการที่ต้องการ speed มากกว่า correctness

2.4.5 DDD ไม่ใช่

- ไม่ใช่ framework
- ไม่ใช่สถาปัตยกรรม (แต่ส่งผลต่อสถาปัตยกรรม)
- ไม่ใช่ silver bullet
- ไม่ใช่แค่การใช้ Entity/Value Object

2.5 ข้อดีของบท

- สร้างความเข้าใจพื้นฐานที่ถูกต้อง
- ทำให้เห็นคุณค่าและข้อจำกัด
- เตรียมพื้นฐานสำหรับบทถัดไป

2.6 ข้อเสีย/ข้อจำกัด

- เป็นภาพรวม ยังไม่ลงรายละเอียด
- ตัวอย่างเป็นระดับแนวคิด

2.7 ข้อควรระวัง

- อย่าใช้ DDD กับทุกระบบ

- อย่าคิดว่า DDD เป็นเรื่องเทคนิคเท่านั้น
- อย่าละเลยการมีส่วนร่วมของผู้เชี่ยวชาญธุรกิจ

2.8 ตัวอย่าง

ระบบประกันภัย:

- Business logic ซับซ้อน (การคำนวณเบี้ย, การอนุมัติสินไหม)
- มีผู้เชี่ยวชาญธุรกิจที่พร้อมร่วมมือ
- ต้องเปลี่ยนแปลงตามกฎหมายและผลิตภัณฑ์ใหม่
- **เหมาะสมกับ DDD**

ระบบจัดการไฟล์:

- CRUD ง่าย ๆ
- ไม่มี business logic ซับซ้อน
- **ไม่เหมาะสมกับ DDD** (ใช้ simple architecture พอ)

2.9 การประยุกต์ใช้

- ใช้ประเมินว่าระบบเหมาะสมกับ DDD หรือไม่
- ใช้สื่อสารกับผู้มีส่วนได้ส่วนเสีย
- ใช้เป็นจุดเริ่มต้นของโครงการ

2.10 สรุป/ผลลัพธ์

ผู้อ่านเข้าใจ DDD และเห็นคุณค่าและข้อจำกัด

2.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 3
- ไปสู่เล่ม 1 (สถาปัตยกรรมพื้นฐาน)
- ไปสู่เล่ม 4 (Clean Architecture)

บทที่ 3: Strategic DDD vs Tactical DDD

3.1 เรื่อง

การแบ่ง DDD เป็นสองระดับ — Strategic (กลยุทธ์) และ Tactical (ยุทธวิธี)

3.2 วัตถุประสงค์

- อธิบายความแตกต่างของสองระดับ

- ทำให้เห็นว่าทั้งสองทำงานร่วมกันอย่างไร
- เตรียมพื้นฐานสำหรับส่วนที่ 2 และ 3

3.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- แยกแยะ Strategic กับ Tactical DDD ได้
- เข้าใจว่าทั้งสอง complement กัน
- รู้ว่าต้องเรียนรู้อะไรในแต่ละส่วน

3.4 หัวข้อหลัก

3.4.1 Strategic DDD

จุดเน้น: ภาพรวมของระบบและขอบเขต

เครื่องมือ:

- Domain / Subdomains
- Bounded Contexts
- Ubiquitous Language
- Context Mapping

คำถามที่ตอบ:

- ระบบแบ่งเป็นส่วนอะไรบ้าง
- แต่ละส่วนมีขอบเขตอย่างไร
- ส่วนต่าง ๆ สัมพันธ์กันอย่างไร
- ภาษาอะไรใช้ในแต่ละส่วน

3.4.2 Tactical DDD

จุดเน้น: โมเดลภายใน bounded context

เครื่องมือ:

- Entities
- Value Objects
- Aggregates
- Repositories
- Domain Events
- Domain Services
- Factories

คำถามที่ตอบ:

- โมเดลภายใน context หน้าตาอย่างไร
- อะไรคือ entity, value object, aggregate
- ข้อมูลถูกจัดเก็บและเรียกคืนอย่างไร
- เหตุการณ์อะไรเกิดขึ้นในโดเมน

3.4.3 ความสัมพันธ์

- **Strategic ก่อน Tactical** — ต้องรู้ขอบเขตก่อนออกแบบภายใน
- **Strategic ไม่มี Tactical = แผนที่ไม่มีเมือง**
- **Tactical ไม่มี Strategic = เมืองที่ไม่มีแผนที่**
- ทั้งสองจำเป็น

3.4.4 ลำดับการทำงาน

1. **Strategic:** ระบุ domain, subdomains, bounded contexts
2. **Strategic:** สร้าง ubiquitous language
3. **Strategic:** ทำ context mapping
4. **Tactical:** ออกแบบโมเดลภายในแต่ละ context
5. **Tactical:** ออกแบบ aggregate, repository, event
6. **Architecture:** จัดโครงสร้างโค้ดให้สะท้อน

3.4.5 ตัวอย่าง

ระบบ e-commerce:

- **Strategic:** แบ่งเป็น bounded contexts: Sales, Inventory, Shipping, Payment
- **Tactical:** ใน Sales context ออกแบบ Aggregate: Order (มี OrderItem เป็น entity, Money เป็น value object)
- **Architecture:** จัดโครงสร้างโค้ดให้ Sales context มี domain, application, infrastructure

3.5 ข้อดีของบท

- ทำให้เห็นภาพรวมของ DDD
- เตรียมพื้นฐานสำหรับส่วนที่ 2 และ 3
- ลดความสับสน

3.6 ข้อเสีย/ข้อจำกัด

- เป็นภาพรวม ยังไม่ลงรายละเอียด
- ต้องอ่านส่วนที่ 2 และ 3 เพื่อความเข้าใจเต็ม

3.7 ข้อควรระวัง

- อย่าข้าม Strategic ไป Tactical ทันที
- อย่าละเลย Tactical หลังจากทำ Strategic

- อย่าคิดว่าทั้งสองแยกจากกัน

3.8 ตัวอย่าง

ทีมที่ทำ Strategic DDD อย่างเดียว:

- มี bounded context ชัดเจน
- แต่โมเดลภายในไม่สะท้อนธุรกิจ
- ผลลัพธ์: ขอบเขตดีแต่โมเดลไม่ดี

ทีมที่ทำ Tactical DDD อย่างเดียว:

- มี entity, value object, aggregate
- แต่ไม่มีขอบเขตชัดเจน
- ผลลัพธ์: โมเดลดีแต่กระจัดกระจาย

3.9 การประยุกต์ใช้

- ใช้เป็นแผนที่นำทางก่อนอ่านส่วนที่ 2 และ 3
- ใช้สื่อสารกับทีมเรื่องลำดับการทำงาน
- ใช้ใน workshop

3.10 สรุป/ผลลัพธ์

ผู้อ่านแยกแยะ Strategic กับ Tactical DDD ได้ และเห็นความสัมพันธ์

3.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่ส่วนที่ 2 (Strategic DDD)
- ไปสู่ส่วนที่ 3 (Tactical DDD)
- ไปสู่เล่ม 2 (Microservices — ใช้ Strategic DDD กำหนดขอบเขต)

ส่วนที่ 2: Strategic DDD (การออกแบบระดับกลยุทธ์)

บทที่ 4: Domain และ Subdomains

4.1 เรื่อง

Domain, Subdomain และการแบ่งโดเมนธุรกิจออกเป็นส่วนย่อย

4.2 วัตถุประสงค์

- อธิบายความหมายของ domain และ subdomain
- ทำให้เห็นวิธีการแบ่งโดเมน
- เตรียมพื้นฐานสำหรับ bounded context

4.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ระบุ domain ของระบบได้
- แบ่ง subdomain ได้
- จำแนกประเภทของ subdomain

4.4 หัวข้อหลัก

4.4.1 Domain คืออะไร

Domain คือ **ขอบเขตของธุรกิจ** ที่ระบบรองรับ เช่น:

- ระบบธนาคาร → domain: banking
- ระบบประกัน → domain: insurance
- ระบบ e-commerce → domain: e-commerce

Domain ไม่ใช่ software domain แต่เป็น business domain

4.4.2 Subdomain คืออะไร

Subdomain คือ **ส่วนย่อยของ domain** ที่มีความเชี่ยวชาญเฉพาะ เช่น domain e-commerce แบ่งเป็น:

- Catalog (สินค้า)
- Order (คำสั่งซื้อ)
- Payment (ชำระเงิน)
- Shipping (จัดส่ง)
- Customer (ลูกค้า)

4.4.3 ประเภทของ Subdomain

- **Core Domain** — หัวใจของธุรกิจ, สร้างความได้เปรียบแข่งขัน, ต้องลงทุนมากที่สุด
- **Supporting Subdomain** — สนับสนุน core domain, จำเป็นแต่ไม่ใช่จุดแข่งขัน
- **Generic Subdomain** — งานทั่วไปที่ซื้อ/ใช้ของสำเร็จรูปได้

4.4.4 การจำแนก Subdomain

คำถามที่ช่วยจำแนก:

- Core: สร้างความแตกต่างจากคู่แข่งหรือไม่?
- Supporting: จำเป็นต่อธุรกิจแต่ไม่มีใครซื้อเพราะสิ่งนี้หรือไม่?

- Generic: มี solution สำเร็จรูปหรือไม่?

4.4.5 การลงทุนตามประเภท

- **Core:** ลงทุนสูงสุด, ทีมที่ดีที่สุด, ออกแบบเอง
- **Supporting:** ลงทุนปานกลาง, อาจ outsource
- **Generic:** ลงทุนต่ำสุด, ซื้อ/ใช้ open source

4.4.6 ตัวอย่างการแบ่ง Subdomain

ระบบ e-commerce:

Subdomain	ประเภท	เหตุผล
Product Recommendation	Core	สร้างความแตกต่าง
Order Management	Supporting	จำเป็นแต่ไม่ใช่จุดแข่งขัน
Payment Processing	Generic	ใช้ Payment Gateway สำเร็จรูป
User Authentication	Generic	ใช้ OAuth/social login
Inventory	Supporting	จำเป็น, อาจ outsource

4.5 ข้อดีของบท

- ทำให้เห็นภาพรวมของโดเมน
- ช่วยจัดลำดับการลงทุน
- เตรียมพื้นฐานสำหรับ bounded context

4.6 ข้อเสีย/ข้อจำกัด

- การจำแนกขึ้นกับบริบท
- ต้องอาศัยความเข้าใจธุรกิจ

4.7 ข้อควรระวัง

- อย่าแบ่ง subdomain ตามเทคโนโลยี
- อย่าละเลยการจำแนกประเภท
- อย่าลงทุนเท่ากันทุก subdomain

4.8 ตัวอย่าง

ระบบประกันภัย:

- **Core:** Underwriting (การพิจารณารับประกัน)

- **Supporting:** Claims Management (การจัดการสินไหม)
- **Generic:** Document Management (การจัดการเอกสาร), Notification (การแจ้งเตือน)

4.9 การประยุกต์ใช้

- ใช้ใน workshop แบ่ง subdomain
- ใช้จัดลำดับการลงทุน
- ใช้กำหนดทีม

4.10 สรุป/ผลลัพธ์

ผู้อ่านระบุ domain และแบ่ง subdomain ได้

4.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 5 (Bounded Context)
- ไปสู่เล่ม 2 (Microservices — กำหนดขอบเขตบริการ)

บทที่ 5: Bounded Contexts

5.1 เรื่อง

Bounded Context — ขอบเขตของโมเดลและภาษา

5.2 วัตถุประสงค์

- อธิบายความหมายของ bounded context
- ทำให้เห็นความสำคัญของขอบเขต
- เตรียมพื้นฐานสำหรับ context mapping

5.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ระบุ bounded context ได้
- แยกความแตกต่างระหว่าง subdomain กับ bounded context
- ออกแบบ bounded context ได้

5.4 หัวข้อหลัก

5.4.1 Bounded Context คืออะไร

Bounded Context คือ **ขอบเขตที่โมเดลหนึ่ง ๆ ถูกนำไปใช้** ภายในขอบเขตนี้:

- มี ubiquitous language ของตนเอง
- มีโมเดลของตนเอง
- คำเดียวกันอาจมีความหมายต่างกันใน context อื่น

5.4.2 Subdomain vs Bounded Context

- **Subdomain** — แนวคิดทางธุรกิจ (problem space)
- **Bounded Context** — แนวคิดทาง solution (solution space)

โดยทั่วไป 1 subdomain ≈ 1 bounded context แต่ไม่เสมอ:

- subdomain ใหญ่อาจมีหลาย bounded context
- bounded context อาจครอบคลุมหลาย subdomain (ไม่แนะนำ)

5.4.3 ตัวอย่างคำที่มีความหมายต่างกัน

คำว่า "Customer":

- ใน **Sales context** — ลูกค้าที่ซื้อสินค้า
- ใน **Support context** — ผู้ที่ติดต่อขอความช่วยเหลือ
- ใน **Billing context** — ผู้ที่ต้องชำระเงิน

แต่ละ context มีโมเดล Customer ของตนเอง

5.4.4 การกำหนด Bounded Context

ปัจจัยที่พิจารณา:

- ภาษาและความหมายของคำ
- โมเดลและ business rules
- ทีมที่รับผิดชอบ
- ขอบเขตข้อมูล
- ขอบเขตการเปลี่ยนแปลง

เครื่องมือ:

- Event Storming
- Domain Storytelling
- Context Mapping

5.4.5 ขอบเขตที่ดี vs ไม่ดี

ขอบเขตที่ดี:

- มี ubiquitous language ชัดเจน
- มีโมเดลที่เป็นเอกภาพ
- มีทีมรับผิดชอบชัดเจน
- เปลี่ยนแปลงได้โดยไม่กระทบ context อื่นมาก

ขอบเขตที่ไม่ดี:

- ภาษาและความหมายขัดแย้งกัน
- โมเดลไม่เป็นเอกภาพ
- หลายทีมรับผิดชอบ
- coupling สูงกับ context อื่น

5.5 ข้อดีของบท

- ทำให้เห็นความสำคัญของขอบเขต
- เตรียมพื้นฐานสำหรับ context mapping
- เชื่อมโยงกับ ubiquitous language

5.6 ข้อเสีย/ข้อจำกัด

- การกำหนดขอบเขตต้องอาศัยความเข้าใจธุรกิจ
- ไม่มีสูตรสำเร็จ

5.7 ข้อควรระวัง

- อย่าแบ่ง bounded context ตามเทคโนโลยี
- อย่าให้ bounded context ใหญ่เกินไป
- อย่าละเลย ubiquitous language

5.8 ตัวอย่าง

ระบบ e-commerce:

- **Sales Context** — Order, Cart, Customer (ผู้ซื้อ)
- **Inventory Context** — Product, Stock, Warehouse
- **Shipping Context** — Shipment, Carrier, Address
- **Payment Context** — Payment, Transaction, Refund

แต่ละ context มีโมเดล Customer/Product ของตนเอง (ไม่แชร์)

5.9 การประยุกต์ใช้

- ใช้ Event Storming เพื่อระบุ bounded context
- ใช้ Bounded Context Canvas (ภาคผนวก B)
- ใช้ใน architecture review

5.10 สรุป/ผลลัพธ์

ผู้อ่านระบุ bounded context ได้ และเข้าใจความแตกต่างจาก subdomain

5.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 6–7
- ไปสู่เล่ม 2 (Microservices — 1 bounded context ≈ 1 **บริการ**)

บทที่ 6: Ubiquitous Language

6.1 เรื่อง

Ubiquitous Language — **ภาษากลางที่ทีมเทคนิคและธุรกิจใช้ร่วมกัน**

6.2 วัตถุประสงค์

- อธิบายความหมายและความสำคัญ
- ทำให้เห็นวิธีสร้างและรักษา
- เตรียมพื้นฐานสำหรับโมเดลที่ดี

6.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- สร้าง ubiquitous language ได้
- ใช้ภาษาให้สอดคล้องในโค้ดและเอกสาร
- รักษาภาษาให้คงอยู่

6.4 หัวข้อหลัก

6.4.1 ความหมาย

Ubiquitous Language **คือ *ภาษากลาง*** ที่:

- ใช้ร่วมกันระหว่างทีมเทคนิคและผู้เชี่ยวชาญธุรกิจ
- ปรากฏในโค้ด, เอกสาร, การสนทนา
- มีความหมายเดียวกันทุกที่ภายใน bounded context

6.4.2 ความสำคัญ

- ลดการเข้าใจผิด
- ทำให้โค้ดสื่อความหมายทางธุรกิจ
- ทำให้การสื่อสารมีประสิทธิภาพ
- เป็นพื้นฐานของโมเดลที่ดี

6.4.3 การสร้าง Ubiquitous Language

1. **พูดคุยกับผู้เชี่ยวชาญธุรกิจ** — ฟังคำที่ใช้จริง
2. **จดคำศัพท์** — สร้าง glossary
3. **ตกลงความหมาย** — ให้ทุกคนเห็นตรงกัน
4. **ใช้ในโค้ด** — ตั้งชื่อ class, method, variable ตามภาษา
5. **ใช้ในเอกสาร** — เขียนเอกสารด้วยภาษาเดียวกัน
6. **ทบทวนและปรับ** — ภาษา evolve ได้

6.4.4 ตัวอย่าง

Ubiquitous Language ของ Sales Context:

คำ	ความหมาย
Order	คำสั่งซื้อของลูกค้า
Cart	ตะกร้าสินค้าก่อนสั่งซื้อ
Line Item	รายการสินค้าใน order
Checkout	กระบวนการยืนยัน order
Fulfillment	กระบวนการจัดส่ง

โค้ดใช้ชื่อเหล่านี้:

```
class Order {
    private List<LineItem> lineItems;
    void checkout() { ... }
}
```

6.4.5 การรักษา Ubiquitous Language

- ใช้ใน code review
- ใช้ในเอกสาร
- ทบทวนเมื่อธุรกิจเปลี่ยน
- หลีกเลี่ยงคำพ้องความหมายที่ไม่จำเป็น
- หลีกเลี่ยงคำทางเทคนิคในโดเมน

6.4.6 ข้อผิดพลาดที่พบบ่อย

- ใช้คำต่างกันในโค้ดและเอกสาร
- ใช้คำทางเทคนิคแทนคำทางธุรกิจ
- ปล่อยให้ภาษาเปลี่ยนโดยไม่ทบทวน

- ใช้ภาษาเดียวกันข้าม bounded context (ที่ความหมายต่างกัน)

6.5 ข้อดีของบท

- ทำให้เห็นความสำคัญของภาษา
- ให้แนวทางสร้างและรักษา
- เตรียมพื้นฐานสำหรับโมเดลที่ดี

6.6 ข้อเสีย/ข้อจำกัด

- ต้องอาศัยการมีส่วนร่วมของผู้เชี่ยวชาญธุรกิจ
- ต้องใช้เวลาในการสร้าง

6.7 ข้อควรระวัง

- อย่าละเลยการมีส่วนร่วมของผู้เชี่ยวชาญธุรกิจ
- อย่าใช้ภาษาเดียวกันข้าม bounded context
- อย่าปล่อยให้ภาษาเปลี่ยนโดยไม่ทบทวน

6.8 ตัวอย่าง

ทีมพัฒนาระบบประกัน:

- ผู้เชี่ยวชาญธุรกิจใช้คำว่า “Policy”, “Claim”, “Premium”, “Coverage”
- ทีมเทคนิคใช้คำเดียวกันในโค้ด
- เอกสารใช้คำเดียวกัน
- เมื่อธุรกิจเปลี่ยนความหมายของ “Premium” ทีมทบทวนและปรับ

6.9 การประยุกต์ใช้

- ใช้สร้าง glossary (ภาคผนวก C)
- ใช้ใน code review
- ใช้ในเอกสาร

6.10 สรุป/ผลลัพธ์

ผู้อ่านสร้างและรักษา ubiquitous language ได้

6.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 7
- ไปสู่บทที่ 15 (โครงสร้างโค้ดที่สะท้อนโดเมน)

บทที่ 7: Context Mapping

7.1 เรื่อง

Context Mapping — การ map ความสัมพันธ์ระหว่าง bounded context

7.2 วัตถุประสงค์

- อธิบายความหมายและความสำคัญ
- ทำให้เห็นรูปแบบความสัมพันธ์
- เตรียมพื้นฐานสำหรับการออกแบบระบบ

7.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- สร้าง context map ได้
- ระบุรูปแบบความสัมพันธ์ได้
- จัดการความสัมพันธ์ระหว่างทีม

7.4 หัวข้อหลัก

7.4.1 Context Map คืออะไร

Context Map คือ **แผนที่** ที่แสดง:

- bounded context ทั้งหมด
- ความสัมพันธ์ระหว่าง context
- ทิศทางของ dependency
- รูปแบบความสัมพันธ์

7.4.2 รูปแบบความสัมพันธ์

- **Partnership** — สองทีมร่วมมือกัน, เปลี่ยนแปลงพร้อมกัน
- **Shared Kernel** — แชร์โมเดลบางส่วน (ระวัง coupling)
- **Customer/Supplier** — downstream ขึ้นกับ upstream
- **Conformist** — downstream ยอมรับโมเดลของ upstream
- **Anticorruption Layer (ACL)** — downstream แปลงโมเดลของ upstream
- **Separate Ways** — ไม่มีความสัมพันธ์
- **Open Host Service** — upstream เปิด API สำหรับหลาย downstream
- **Published Language** — ใช้ภาษากลางที่ตกลงกัน

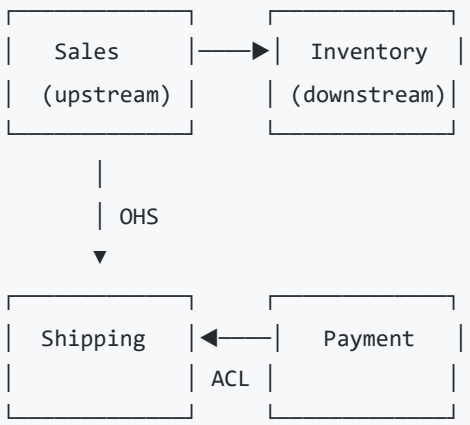
7.4.3 การเลือกความสัมพันธ์

สถานการณ์	แนะนำ
ทีมเดียวกัน	Shared Kernel (ระวัง)
ต้องการอิสระ	Separate Ways
upstream เปลี่ยนแปลงบ่อย	Anticorruption Layer
upstream stable	Conformist
หลาย downstream	Open Host Service

7.4.4 Anticorruption Layer (ACL)

- ชั้นแปลงระหว่าง context
- ป้องกันโมเดลของ upstream รั่วไหล
- ทำให้ downstream มีโมเดลของตนเอง
- **ข้อดี:** อิสระ
- **ข้อเสีย:** ซับซ้อนขึ้น

7.4.5 ตัวอย่าง Context Map



7.4.6 การจัดการความสัมพันธ์ระหว่างทีม

- ตกลงรูปแบบความสัมพันธ์
- กำหนด contract
- กำหนด communication
- ทบทวนเป็นระยะ

7.5 ข้อดีของบท

- ทำให้เห็นภาพรวมความสัมพันธ์
- ให้เครื่องมือจัดการ dependency
- เตรียมพื้นฐานสำหรับการออกแบบระบบ

7.6 ข้อเสีย/ข้อจำกัด

- ต้องอาศัยความเข้าใจธุรกิจและทีม
- ต้องทบทวนเป็นระยะ

7.7 ข้อควรระวัง

- อย่าละเลย context mapping
- อย่าใช้ Shared Kernel โดยไม่ระวัง
- อย่าละเลย ACL เมื่อจำเป็น
- อย่าคิดว่า context map เป็นเรื่องถาวร

7.8 ตัวอย่าง

ระบบ e-commerce:

- **Sales** → **Inventory:** Customer/Supplier (Sales เป็น upstream)
- **Sales** → **Payment:** Open Host Service (Payment เปิด API)
- **Shipping** → **Payment:** Anticorruption Layer (Shipping แปลงโมเดล Payment)

7.9 การประยุกต์ใช้

- ใช้ Context Map Template (ภาคผนวก D)
- ใช้ใน architecture review
- ใช้สื่อสารกับทีม

7.10 สรุป/ผลลัพธ์

ผู้อ่านสร้าง context map ได้ และเลือกรูปแบบความสัมพันธ์ได้

7.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 8
- ไปสู่เล่ม 2 (Microservices — communication pattern)

บทที่ 8: การเชื่อม Strategic สู่ Tactical

8.1 เรื่อง

การเชื่อมจาก Strategic DDD (ขอบเขต) สู่ Tactical DDD (โมเดลภายใน)

8.2 วัตถุประสงค์

- ทำให้เห็นลำดับการทำงาน
- เตรียมพื้นฐานสำหรับส่วนที่ 3
- ลดช่องว่างระหว่างสองระดับ

8.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- เชื่อม Strategic สู่ Tactical ได้
- วางแผนการออกแบบได้
- รู้ว่าต้องทำอะไรต่อ

8.4 หัวข้อหลัก

8.4.1 ลำดับการทำงาน

1. **Strategic:** ระบุ domain และ subdomain
2. **Strategic:** กำหนด bounded context
3. **Strategic:** สร้าง ubiquitous language
4. **Strategic:** ทำ context mapping
5. **Tactical:** ออกแบบโมเดลภายในแต่ละ context
6. **Architecture:** จัดโครงสร้างโค้ด

8.4.2 การเลือก Context ที่จะเริ่ม

- เริ่มจาก **Core Domain** — สำคัญที่สุด
- หรือเริ่มจาก context ที่มีความเสี่ยงต่ำ — เพื่อเรียนรู้
- พิจารณาความพร้อมของทีม

8.4.3 การเตรียมความพร้อม

- มี ubiquitous language
- มีผู้เชี่ยวชาญธุรกิจ
- มีทีมที่รับผิดชอบ
- มีความเข้าใจ business rules

8.4.4 ตัวอย่าง

ระบบ e-commerce:

- **Strategic:** เลือก Sales Context เป็น Core Domain
- **Tactical:** ออกแบบ Order Aggregate, Customer Entity, Money Value Object
- **Architecture:** จัดโครงสร้าง sales/domain, sales/application, sales/infrastructure

8.5 ข้อดีของบท

- ทำให้เห็นลำดับการทำงาน
- เตรียมพื้นฐานสำหรับส่วนที่ 3
- ลดช่องว่างระหว่างสองระดับ

8.6 ข้อเสีย/ข้อจำกัด

- เป็นบทเชื่อม ไม่ได้ลงรายละเอียด
- ต้องอ่านส่วนที่ 3 เพื่อความเข้าใจเต็ม

8.7 ข้อควรระวัง

- อย่าข้าม Strategic ไป Tactical ทันที
- อย่าละเลย ubiquitous language
- อย่าละเลยการมีส่วนร่วมของผู้เชี่ยวชาญธุรกิจ

8.8 ตัวอย่าง

ทีมที่ทำ Strategic แล้ว:

- มี bounded context: Sales, Inventory, Shipping
- มี ubiquitous language ของแต่ละ context
- มี context map
- **ต่อไป:** เลือก Sales context และเริ่ม Tactical DDD

8.9 การประยุกต์ใช้

- ใช้เป็นแผนที่นำทางก่อนอ่านส่วนที่ 3
- ใช้ใน workshop
- ใช้สื่อสารกับทีม

8.10 สรุป/ผลลัพธ์

ผู้อ่านเชื่อม Strategic สู่ Tactical ได้ และพร้อมสำหรับส่วนที่ 3

8.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่ส่วนที่ 3 (Tactical DDD)
- ไปสู่บทที่ 15 (โครงสร้างโค้ด)

ส่วนที่ 3: Tactical DDD (การออกแบบระดับยุทธวิธี)

บทที่ 9: Entities

9.1 เรื่อง

Entity — วัตถุที่มีIdentity และวงจรชีวิต

9.2 วัตถุประสงค์

- อธิบายความหมายของ entity
- ทำให้เห็นความแตกต่างจาก value object
- เตรียมพื้นฐานสำหรับ aggregate

9.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ระบุ entity ได้
- ออกแบบ entity ได้
- แยกแยะ entity จาก value object

9.4 หัวข้อหลัก

9.4.1 Entity คืออะไร

Entity คือวัตถุที่:

- มี **Identity** — ระบุได้ว่าเป็นตัวไหน
- มี **วงจรชีวิต** — เกิด เปลี่ยน แก้
- มี **ความต่อเนื่อง** — แม้ attributes เปลี่ยน แต่ identity เดิม

9.4.2 Identity

Identity อาจเป็น:

- **Natural ID** — เช่น เลขบัตรประชาชน

- **Surrogate ID** — เช่น UUID, auto-increment
- **Composite ID** — หลาย field รวมกัน

9.4.3 Entity vs Value Object

ลักษณะ	Entity	Value Object
Identity	มี	ไม่มี
Equality	ตาม identity	ตามค่า
Mutability	มัก mutable	immutable
ตัวอย่าง	Order, Customer	Money, Address

9.4.4 การออกแบบ Entity

- ให้ identity ชัดเจน
- เก็บเฉพาะข้อมูลที่จำเป็น
- หลีกเลี่ยง anemic entity (แค่ getter/setter)
- ให้ entity มี behavior ที่สื่อความหมาย

9.4.5 Anemic vs Rich Entity

Anemic:

```
class Order {
    private Long id;
    private List<LineItem> items;
    // getters/setters
}
```

Rich:

```
class Order {
    private OrderId id;
    private List<LineItem> items;

    void addItem(Product product, int quantity) {
        // validation + logic
    }

    Money total() {
        return items.stream()
            .map(LineItem::subtotal)
```

```
            .reduce(Money.ZERO, Money::add);
    }
}
```

9.4.6 ข้อผิดพลาดที่พบบ่อย

- Entity ที่มี behavior น้อยเกินไป
- Identity ที่ไม่ชัดเจน
- Entity ที่ใหญ่เกินไป (god entity)
- Entity ที่ผูกกับ database schema

9.5 ข้อดีของบท

- ทำให้เห็นความสำคัญของ identity
- เตรียมพื้นฐานสำหรับ aggregate
- สอนการออกแบบ entity ที่ดี

9.6 ข้อเสีย/ข้อจำกัด

- ตัวอย่างอาจไม่ครอบคลุมทุกภาษา
- ต้องฝึกฝน

9.7 ข้อควรระวัง

- อย่าสร้าง anemic entity
- อย่าให้ entity ผูกกับ database
- อย่าให้ entity ใหญ่เกินไป

9.8 ตัวอย่าง

```java
public class Order {
    private final OrderId id;
    private final CustomerId customerId;
    private final List<LineItem> items;
    private OrderStatus status;

    public void addItem(Product product, int quantity) {
        if (status != OrderStatus.DRAFT) {
            throw new IllegalStateException("Cannot modify confirmed order");
        }
        items.add(new LineItem(product, quantity));
    }

    public void confirm() {
```

```
        if (items.isEmpty()) {
            throw new IllegalStateException("Cannot confirm empty order");
        }
        this.status = OrderStatus.CONFIRMED;
    }
}
```

9.9 การประยุกต์ใช้

- ใช้ระบุ entity ใน bounded context
- ใช้ใน aggregate design
- ใช้ใน code review

9.10 สรุป/ผลลัพธ์

ผู้อ่านระบุและออกแบบ entity ได้

9.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 10–11
- ไปสู่เล่ม 4 (Entities ใน Clean Architecture)

บทที่ 10: Value Objects

10.1 เรื่อง

Value Object — วัตถุที่ไม่มี identity แต่มีค่า

10.2 วัตถุประสงค์

- อธิบายความหมายของ value object
- ทำให้เห็นประโยชน์
- เตรียมพื้นฐานสำหรับ aggregate

10.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ระบุ value object ได้
- ออกแบบ value object ได้
- ใช้ value object แทน primitive

10.4 หัวข้อหลัก

10.4.1 Value Object คืออะไร

Value Object คือวัตถุที่:

- **ไม่มี identity** — ระบุด้วยค่า
- **Immutable** — เปลี่ยนไม่ได้
- **Equality ตามค่า** — ค่าเท่ากัน = เท่ากัน
- **แทนแนวคิดทางธุรกิจ** — เช่น Money, Address, DateRange

10.4.2 ตัวอย่าง Value Object

- **Money** — จำนวน + สกุลเงิน
- **Address** — บ้านเลขที่ ถนน เมือง รหัสไปรษณีย์
- **DateRange** — วันเริ่ม วันสิ้นสุด
- **Email** — ที่อยู่อีเมลที่ validate แล้ว
- **PhoneNumber** — เบอร์โทรที่ validate แล้ว

10.4.3 Value Object vs Primitive

Primitive:

```
class Customer {
    private String email;  // string ธรรมดา
}
```

Value Object:

```
class Customer {
    private Email email;  // value object
}

class Email {
    private final String value;

    public Email(String value) {
        if (!isValid(value)) {
            throw new IllegalArgumentException("Invalid email");
        }
        this.value = value;
    }
}
```

10.4.4 ประโยชน์

- **Type safety** — ป้องกันการสลับ parameter
- **Validation** — validate ที่จุดสร้าง
- **Immutability** — ลด bug
- **Expressiveness** — สื่อความหมายชัดเจน
- **Behavior** — มี method ที่สื่อความหมาย

10.4.5 การออกแบบ Value Object

- Immutable
- Validate ใน constructor
- Equality ตามค่า
- ไม่มี identity
- มี behavior ที่สื่อความหมาย

10.4.6 ข้อผิดพลาดที่พบบ่อย

- ใช้ primitive แทน value object
- Value object ที่ mutable
- Value object ที่มี identity
- Value object ที่ไม่มี validation

10.5 ข้อดีของบท

- ทำให้เห็นประโยชน์ของ value object
- เตรียมพื้นฐานสำหรับ aggregate
- สอนการออกแบบที่ดี

10.6 ข้อเสีย/ข้อจำกัด

- อาจเพิ่มจำนวน class
- ต้องฝึกฝน

10.7 ข้อควรระวัง

- อย่าใช้ primitive แทน value object
- อย่าให้ value object mutable
- อย่าละเลย validation

10.8 ตัวอย่าง

```
public final class Money {
    private final BigDecimal amount;
```

```
    private final Currency currency;

    public Money(BigDecimal amount, Currency currency) {
        if (amount == null || currency == null) {
            throw new IllegalArgumentException("Amount and currency required");
        }
        this.amount = amount;
        this.currency = currency;
    }

    public Money add(Money other) {
        if (!this.currency.equals(other.currency)) {
            throw new IllegalArgumentException("Cannot add different currencies");
        }
        return new Money(this.amount.add(other.amount), this.currency);
    }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Money)) return false;
        Money other = (Money) o;
        return amount.equals(other.amount) && currency.equals(other.currency);
    }
}
```

10.9 การประยุกต์ใช้

- ใช้ระบุ value object ใน bounded context
- ใช้แทน primitive ที่มีความหมายทางธุรกิจ
- ใช้ใน code review

10.10 สรุป/ผลลัพธ์

ผู้อ่านระบุและออกแบบ value object ได้

10.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 11
- ไปสู่เล่ม 4 (Value Objects ใน Clean Architecture)

บทที่ 11: Aggregates

11.1 เรื่อง

Aggregate — กลุ่มของ entity และ value object ที่มี boundary

11.2 วัตถุประสงค์

- อธิบายความหมายของ aggregate
- ทำให้เห็นความสำคัญของ boundary
- เตรียมพื้นฐานสำหรับ repository

11.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ระบุ aggregate ได้
- ออกแบบ aggregate ได้
- กำหนด aggregate root ได้

11.4 หัวข้อหลัก

11.4.1 Aggregate คืออะไร

Aggregate คือ **กลุ่มของ entity และ value object** ที่:

- มี **Aggregate Root** — entity หลักที่ควบคุม
- มี **Boundary** — ขอบเขตของ consistency
- มี **Transactional consistency** — เปลี่ยนแปลงภายใน aggregate พร้อมกัน
- เข้าถึงจากภายนอกผ่าน aggregate root เท่านั้น

11.4.2 Aggregate Root

- Entity หลักของ aggregate
- เป็นจุดเข้าถึงเดียวจากภายนอก
- รับผิดชอบรักษา invariants
- มี identity

11.4.3 ตัวอย่าง Aggregate

Order Aggregate:

- Root: Order
- Entities: OrderItem
- Value Objects: Money, Address, OrderId

Customer Aggregate:

- Root: Customer
- Value Objects: Email, PhoneNumber, Address

11.4.4 กฎของ Aggregate

1. **Reference by identity** — อ้างถึง aggregate อื่นด้วย identity เท่านั้น
2. **One transaction per aggregate** — หนึ่ง transaction แก้หนึ่ง aggregate
3. **Eventual consistency** — ระหว่าง aggregate ใช้ eventual consistency
4. **Small aggregates** — aggregate ควรเล็ก
5. **Invariants** — aggregate รักษา invariants ของตนเอง

11.4.5 การออกแบบ Aggregate

- เริ่มจาก invariants ที่ต้องรักษา
- กำหนด boundary ที่เล็กที่สุด
- ระบุ aggregate root
- ออกแบบ behavior ที่รักษา invariants
- หลีกเลี่ยง aggregate ที่ใหญ่เกินไป

11.4.6 Aggregate ที่ใหญ่เกินไป vs เล็กเกินไป

ใหญ่เกินไป:

- Performance ต่ำ
- Lock นาน
- Concurrent update ยาก

เล็กเกินไป:

- Invariants กระจัดกระจาย
- ต้องประสานหลาย aggregate
- Consistency ยาก

11.4.7 ตัวอย่าง Aggregate Design

```java
// Aggregate Root
public class Order {
    private final OrderId id;
    private final CustomerId customerId;
    private final List<OrderItem> items;
    private OrderStatus status;

    // Invariant: order ต้องมี item อย่างน้อย 1 ก่อน confirm
    public void confirm() {
```

```
        if (items.isEmpty()) {
            throw new IllegalStateException("Cannot confirm empty order");
        }
        this.status = OrderStatus.CONFIRMED;
    }

    // Invariant: จำนวนต้องเป็นบวก
    public void addItem(Product product, int quantity) {
        if (quantity <= 0) {
            throw new IllegalArgumentException("Quantity must be positive");
        }
        items.add(new OrderItem(product, quantity));
    }
}
```

11.5 ข้อดีของบท

- ทำให้เห็นความสำคัญของ boundary
- เตรียมพื้นฐานสำหรับ repository
- สอนการออกแบบ aggregate ที่ดี

11.6 ข้อเสีย/ข้อจำกัด

- ต้องอาศัยความเข้าใจ invariants
- ต้องฝึกฝน

11.7 ข้อควรระวัง

- อย่าให้ aggregate ใหญ่เกินไป
- อย่าให้ aggregate เล็กเกินไป
- อย่าละเลย invariants
- อย่าอ้างถึง aggregate อื่นด้วย reference (ใช้ identity)

11.8 ตัวอย่าง

Order Aggregate — ประกอบด้วย:

- Order (root)
- OrderItem (entity)
- OrderId, Money (value objects)

ไม่ควร ให้ Order มี reference ตรงไปยัง Customer object — ควรใช้ CustomerId

11.9 การประยุกต์ใช้

- ใช้ Aggregate Design Checklist (ภาคผนวก E)
- ใช้ใน workshop
- ใช้ใน code review

11.10 สรุป/ผลลัพธ์

ผู้อ่านระบุและออกแบบ aggregate ได้

11.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 12–13
- ไปสู่เล่ม 4 (Aggregates ใน Clean Architecture)

บทที่ 12: Repositories

12.1 เรื่อง

Repository — การเข้าถึง aggregate จากภายนอก

12.2 วัตถุประสงค์

- อธิบายความหมายและบทบาท
- ทำให้เห็นความแตกต่างจาก DAO
- เตรียมพื้นฐานสำหรับโครงสร้างโค้ด

12.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ออกแบบ repository ได้
- ใช้ repository กับ aggregate ได้
- แยก repository จาก infrastructure

12.4 หัวข้อหลัก

12.4.1 Repository คืออะไร

Repository คือ **abstraction** สำหรับเข้าถึง aggregate:

- ให้ความรู้สึกเหมือน collection
- ซ่อนรายละเอียดของ persistence
- อยู่ที่ domain layer (interface)

- implement ที่ infrastructure layer

12.4.2 Repository vs DAO

ลักษณะ	Repository	DAO
ระดับ	Domain	Data
หน่วย	Aggregate	Table/Entity
Interface	Domain layer	Data layer
ซ่อน	Persistence	SQL
ภาษา	Ubiquitous	Technical

12.4.3 การออกแบบ Repository

- หนึ่ง repository ต่อหนึ่ง aggregate
- Interface อยู่ที่ domain
- Implementation อยู่ที่ infrastructure
- Method สื่อความหมายทางธุรกิจ
- หลีกเลี่ยง method ที่ผูกกับ SQL

12.4.4 ตัวอย่าง Repository

Interface (domain):

```
public interface OrderRepository {
    Order findById(OrderId id);
    void save(Order order);
    void delete(OrderId id);
    List<Order> findByCustomer(CustomerId customerId);
}
```

Implementation (infrastructure):

```
public class JpaOrderRepository implements OrderRepository {
    private final EntityManager em;

    @Override
    public Order findById(OrderId id) {
        return em.find(OrderEntity.class, id.value())
            .toDomain();
    }
```

```
    @Override
    public void save(Order order) {
        em.persist(OrderEntity.fromDomain(order));
    }
}
```

12.4.5 Repository กับ Aggregate

- Repository จัดการ aggregate ทั้งก้อน
- ไม่มี repository สำหรับ entity ภายใน aggregate
- การโหลด aggregate โหลดทั้งก้อน

12.4.6 ข้อผิดพลาดที่พบบ่อย

- Repository ที่ผูกกับ SQL
- Repository ที่มี method มากเกินไป
- Repository สำหรับ entity ภายใน aggregate
- Repository ที่อยู่ที่ infrastructure layer (interface)

12.5 ข้อดีของบท

- ทำให้เห็นความสำคัญของ abstraction
- เตรียมพื้นฐานสำหรับโครงสร้างโค้ด
- สอนการออกแบบ repository ที่ดี

12.6 ข้อเสีย/ข้อจำกัด

- ต้องอาศัยความเข้าใจ aggregate
- ต้องฝึกฝน

12.7 ข้อควรระวัง

- อย่าให้ repository ผูกกับ SQL
- อย่าสร้าง repository สำหรับ entity ภายใน aggregate
- อย่าให้ interface อยู่ที่ infrastructure

12.8 ตัวอย่าง

```
// Domain
public interface OrderRepository {
    Optional<Order> findById(OrderId id);
    void save(Order order);
}
```

```java
// Application
public class PlaceOrderUseCase {
    private final OrderRepository orders;

    public void execute(PlaceOrderCommand cmd) {
        Order order = Order.create(cmd.customerId());
        // ...
        orders.save(order);
    }
}

// Infrastructure
public class JpaOrderRepository implements OrderRepository {
    // implementation
}
```

12.9 การประยุกต์ใช้

- ใช้ออกแบบ repository ใน bounded context
- ใช้ใน code review
- ใช้ใน architecture review

12.10 สรุป/ผลลัพธ์

ผู้อ่านออกแบบ repository ได้ และแยกจาก infrastructure

12.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 13
- ไปสู่เล่ม 4 (Repositories ใน Clean Architecture)

บทที่ 13: Domain Events

13.1 เรื่อง

Domain Event — เหตุการณ์ทางธุรกิจที่เกิดขึ้นในโดเมน

13.2 วัตถุประสงค์

- อธิบายความหมายและบทบาท
- ทำให้เห็นประโยชน์
- เตรียมพื้นฐานสำหรับ event-driven

13.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ระบุ domain event ได้
- ออกแบบ domain event ได้
- ใช้ domain event ในการสื่อสาร

13.4 หัวข้อหลัก

13.4.1 Domain Event คืออะไร

Domain Event คือ **เหตุการณ์ทางธุรกิจ** ที่:

- เกิดขึ้นในโดเมน
- มีความหมายทางธุรกิจ
- เกิดขึ้นแล้ว (past tense)
- ถูก publish เมื่อเกิด

13.4.2 ตัวอย่าง Domain Event

- `OrderPlaced`
- `OrderConfirmed`
- `PaymentReceived`
- `ShipmentDispatched`
- `CustomerRegistered`

13.4.3 ประโยชน์

- **Decoupling** — ผู้ส่งไม่รู้จักผู้รับ
- **Communication** — สื่อสารระหว่าง aggregate/context
- **Audit trail** — บันทึกเหตุการณ์
- **Integration** — เชื่อมกับระบบอื่น
- **Event Sourcing** — เก็บ events เป็น source of truth

13.4.4 การออกแบบ Domain Event

- ชื่อเป็น past tense
- มีข้อมูลที่จำเป็น
- Immutable
- มี timestamp
- มี event ID

13.4.5 ตัวอย่าง Domain Event

```
public class OrderPlaced {
    private final OrderId orderId;
    private final CustomerId customerId;
    private final Money total;
    private final Instant occurredAt;

    public OrderPlaced(OrderId orderId, CustomerId customerId,
                       Money total, Instant occurredAt) {
        this.orderId = orderId;
        this.customerId = customerId;
        this.total = total;
        this.occurredAt = occurredAt;
    }
}
```

13.4.6 การ Publish Domain Event

- **จาก Aggregate** — aggregate publish เมื่อเกิด
- **จาก Application Service** — service publish หลัง save
- **ผ่าน Event Bus** — ส่งต่อให้ผู้สนใจ

13.4.7 Domain Event กับ Integration Event

- **Domain Event** — ภายใน bounded context
- **Integration Event** — ระหว่าง bounded context
- Integration Event มักสร้างจาก Domain Event

13.5 ข้อดีของบท

- ทำให้เห็นประโยชน์ของ domain event
- เตรียมพื้นฐานสำหรับ event-driven
- สอนการออกแบบที่ดี

13.6 ข้อเสีย/ข้อจำกัด

- ต้องอาศัยความเข้าใจโดเมน
- ต้องระวัง consistency

13.7 ข้อควรระวัง

- อย่าให้ domain event ผูกกับ infrastructure
- อย่าละเลย idempotency
- อย่าละเลย ordering

- อย่าให้ domain event ใหญ่เกินไป

13.8 ตัวอย่าง

```
public class Order {
    private final List<DomainEvent> events = new ArrayList<>();

    public void confirm() {
        if (items.isEmpty()) {
            throw new IllegalStateException("Cannot confirm empty order");
        }
        this.status = OrderStatus.CONFIRMED;
        this.events.add(new OrderConfirmed(id, Instant.now()));
    }

    public List<DomainEvent> pullEvents() {
        List<DomainEvent> result = new ArrayList<>(events);
        events.clear();
        return result;
    }
}
```

13.9 การประยุกต์ใช้

- ใช้ Domain Event Catalog (ภาคผนวก F)
- ใช้ใน event-driven architecture
- ใช้ใน integration

13.10 สรุป/ผลลัพธ์

ผู้อ่านระบุและออกแบบ domain event ได้

13.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 14
- ไปสู่เล่ม 2 (Event-driven communication)

บทที่ 14: Domain Services และ Factories

14.1 เรื่อง

Domain Service และ Factory — เครื่องมือเสริมใน Tactical DDD

14.2 วัตถุประสงค์

- อธิบายความหมายและบทบาท
- ทำให้เห็นเมื่อใดควรใช้
- เตรียมพื้นฐานสำหรับการออกแบบที่ครบถ้วน

14.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ระบุเมื่อใดควรใช้ domain service
- ระบุเมื่อใดควรใช้ factory
- ออกแบบได้อย่างเหมาะสม

14.4 หัวข้อหลัก

14.4.1 Domain Service คืออะไร

Domain Service คือ **operation ที่ไม่เป็นธรรมชาติของ entity หรือ value object**:

- มี business logic
- ไม่มี state
- ทำงานกับหลาย aggregate
- อยู่ใน domain layer

14.4.2 เมื่อใดควรใช้ Domain Service

- Logic ที่เกี่ยวข้องกับหลาย entity/aggregate
- Logic ที่ไม่เป็นธรรมชาติของ entity ใด entity หนึ่ง
- Logic ที่ต้องประสานหลายอย่าง

14.4.3 ตัวอย่าง Domain Service

```
public class TransferService {
    public void transfer(Account from, Account to, Money amount) {
        if (!from.canWithdraw(amount)) {
            throw new InsufficientFundsException();
        }
        from.withdraw(amount);
        to.deposit(amount);
    }
}
```

14.4.4 Factory คืออะไร

Factory คือ **Abstraction สำหรับสร้าง object ที่ซับซ้อน**:

- ซ่อน complexity ของการสร้าง
- รักษา invariants
- อยู่ใน domain layer

14.4.5 เมื่อใดควรใช้ Factory

- การสร้าง object ซับซ้อน
- ต้องรักษา invariants หลายอย่าง
- ต้องเลือก implementation
- ต้องประกอบหลายส่วน

14.4.6 ตัวอย่าง Factory

```
public class OrderFactory {
    public Order createOrder(CustomerId customerId, List<OrderLine> lines) {
        if (lines.isEmpty()) {
            throw new IllegalArgumentException("Order must have lines");
        }
        OrderId id = OrderId.generate();
        return new Order(id, customerId, lines);
    }
}
```

14.4.7 Domain Service vs Application Service

ลักษณะ	Domain Service	Application Service
ระดับ	Domain	Application
Business logic	มี	orchestration
State	ไม่มี	ไม่มี
Transaction	ไม่จัดการ	จัดการ
ตัวอย่าง	TransferService	PlaceOrderUseCase

14.5 ข้อดีของบท

- ทำให้เห็นเครื่องมือเสริมที่จำเป็น
- เตรียมพื้นฐานสำหรับการออกแบบที่ครบถ้วน
- ลดความสับสนระหว่าง service types

14.6 ข้อเสีย/ข้อจำกัด

- ต้องอาศัยความเข้าใจโดเมน
- ใช้มากเกินไปอาจทำให้ logic กระจัดกระจาย

14.7 ข้อควรระวัง

- อย่าใช้ domain service แทน entity behavior
- อย่าให้ domain service มี state
- อย่าสร้าง factory โดยไม่จำเป็น

14.8 ตัวอย่าง

```
// Domain Service
public class PricingService {
    public Money calculatePrice(Order order, DiscountPolicy policy) {
        Money subtotal = order.subtotal();
        return policy.apply(subtotal);
    }
}

// Factory
public class CustomerFactory {
    public Customer createCustomer(String name, String email) {
        return new Customer(
            CustomerId.generate(),
            new CustomerName(name),
            new Email(email)
        );
    }
}
```

14.9 การประยุกต์ใช้

- ใช้ระบุเมื่อใดควรใช้ domain service
- ใช้ระบุเมื่อใดควรใช้ factory
- ใช้ใน code review

14.10 สรุป/ผลลัพธ์

ผู้อ่านระบุและออกแบบ domain service และ factory ได้

14.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 15

- ไปสู่เล่ม 4 (Application Services)

ส่วนที่ 4: สถาปัตยกรรมที่สะท้อนโดเมน

บทที่ 15: โครงสร้างโค้ดที่สะท้อนโดเมน

15.1 เรื่อง

การจัดโครงสร้างโค้ดให้สะท้อนโดเมนและ bounded context

15.2 วัตถุประสงค์

- ทำให้เห็นความสำคัญของโครงสร้าง
- สอนการจัดโครงสร้างที่สะท้อนโดเมน
- เตรียมพื้นฐานสำหรับบทที่ 16

15.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- จัดโครงสร้างโค้ดที่สะท้อนโดเมน
- ใช้ ubiquitous language ในโครงสร้าง
- หลีกเลี่ยงโครงสร้างตามเทคโนโลยี

15.4 หัวข้อหลัก

15.4.1 หลักการ

- **Structure by domain, not by technology**
- **ชื่อ package สื่อ ubiquitous language**
- **แยก bounded context ชัดเจน**
- **Domain อยู่ศูนย์กลาง**

15.4.2 ตัวอย่างโครงสร้าง

```
src/
├── sales/                    # Bounded Context
│   ├── domain/
│   │   ├── model/
│   │   │   ├── Order.java
```

```
│   │   │   ├── OrderItem.java
│   │   │   ├── OrderId.java
│   │   │   └── Money.java
│   │   ├── repository/
│   │   │   └── OrderRepository.java
│   │   └── event/
│   │       └── OrderPlaced.java
│   ├── application/
│   │   └── PlaceOrderUseCase.java
│   ├── infrastructure/
│   │   └── JpaOrderRepository.java
│   └── api/
│       └── OrderController.java
├── inventory/                # Bounded Context
│   ├── domain/
│   ├── application/
│   ├── infrastructure/
│   └── api/
└── shipping/                 # Bounded Context
    ├── domain/
    ├── application/
    ├── infrastructure/
    └── api/
```

15.4.3 การตั้งชื่อ

- ใช้ ubiquitous language
- หลีกเลี่ยงชื่อทางเทคนิค (utils, helpers)
- ใช้ชื่อที่สื่อความหมายทางธุรกิจ

15.4.4 การแยก Bounded Context

- แต่ละ context มี package ของตนเอง
- dependency ระหว่าง context ผ่าน interface/event
- หลีกเลี่ยงการ import ข้าม context โดยตรง

15.4.5 ข้อผิดพลาดที่พบบ่อย

- จัดโครงสร้างตามเทคโนโลยี (controllers/, services/, repositories/)
- ใช้ชื่อ package ทางเทคนิค
- ผสม bounded context ใน package เดียว
- Domain ผูกกับ framework

15.5 ข้อดีของบท

- ทำให้เห็นความสำคัญของโครงสร้าง

- ให้แนวทางปฏิบัติ
- เตรียมพื้นฐานสำหรับบทที่ 16

15.6 ข้อเสีย/ข้อจำกัด

- ต้องปรับตามภาษาและ framework
- ต้องฝึกฝน

15.7 ข้อควรระวัง

- อย่าจัดโครงสร้างตามเทคโนโลยี
- อย่าใช้ชื่อทางเทคนิค
- อย่าผสม bounded context

15.8 ตัวอย่าง

ไม่ดี:

```
src/
├── controllers/
├── services/
├── repositories/
└── models/
```

ดี:

```
src/
├── sales/
│   ├── domain/
│   ├── application/
│   ├── infrastructure/
│   └── api/
└── inventory/
    ├── domain/
    ├── application/
    ├── infrastructure/
    └── api/
```

15.9 การประยุกต์ใช้

- ใช้จัดโครงสร้างโปรเจกต์ใหม่
- ใช้ refactor โปรเจกต์เดิม
- ใช้ใน code review

15.10 สรุป/ผลลัพธ์

ผู้อ่านจัดโครงสร้างโค้ดที่สะท้อนโดเมนได้

15.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 16
- ไปสู่เล่ม 1 (Package architecture)
- ไปสู่เล่ม 4 (Clean Architecture)

บทที่ 16: Reflecting Architecture and Domain in Code

16.1 เรื่อง

การทำให้โค้ดสะท้อนทั้งสถาปัตยกรรมและโดเมน

16.2 วัตถุประสงค์

- รวมแนวคิด architecture และ domain
- ทำให้โค้ดสื่อความหมายทั้งสองมิติ
- เตรียมพื้นฐานสำหรับ Hexagonal/Clean

16.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ออกแบบโครงสร้างที่สะท้อนทั้งสองมิติ
- หลีกเลี่ยงโครงสร้างตาม framework
- รักษา dependency rule

16.4 หัวข้อหลัก

16.4.1 สองมิติ

- **Domain dimension** — bounded context, ubiquitous language
- **Architecture dimension** — layer, dependency rule

16.4.2 การรวมสองมิติ

```
sales/                          # Domain dimension
├── domain/                     # Architecture dimension
│   ├── model/
│   ├── repository/
│   └── event/
```

```
├── application/
├── infrastructure/
└── api/
```

16.4.3 Dependency Rule

- Domain ไม่รู้จัก infrastructure
- Infrastructure implement interface จาก domain
- Application orchestrate domain
- API เรียก application

16.4.4 การหลีกเลี่ยง Framework

- Domain ไม่ผูกกับ framework
- ใช้ framework เป็น infrastructure
- ใช้ dependency injection

16.4.5 ตัวอย่าง

```java
// Domain — ไม่มี framework
public class Order {
    private final OrderId id;
    private final List<OrderItem> items;

    public void confirm() {
        if (items.isEmpty()) {
            throw new IllegalStateException("Empty order");
        }
    }
}

// Infrastructure — ใช้ framework
@Entity
public class OrderEntity {
    @Id private String id;
    // JPA annotations

    public Order toDomain() { ... }
    public static OrderEntity fromDomain(Order order) { ... }
}
```

16.5 ข้อดีของบท

- ทำให้โค้ดสื่อความหมายทั้งสองมิติ
- ลด coupling กับ framework

- เตรียมพื้นฐานสำหรับ Hexagonal/Clean

16.6 ข้อเสีย/ข้อจำกัด

- ต้องปรับโครงสร้างเดิมมาก
- ต้องฝึกฝน

16.7 ข้อควรระวัง

- อย่าให้ domain ผูกกับ framework
- อย่าละเลย dependency rule
- อย่าพยายามสะท้อนทุกอย่างจนซับซ้อนเกิน

16.8 ตัวอย่าง

```
// Application Service
public class PlaceOrderUseCase {
    private final OrderRepository orders;
    private final EventPublisher events;

    @Transactional
    public OrderId execute(PlaceOrderCommand cmd) {
        Order order = Order.create(cmd.customerId(), cmd.items());
        orders.save(order);
        order.pullEvents().forEach(events::publish);
        return order.id();
    }
}
```

16.9 การประยุกต์ใช้

- ใช้ปรับโครงสร้างก่อนทำ Hexagonal/Clean
- ใช้ใน code review
- ใช้ใน architecture review

16.10 สรุป/ผลลัพธ์

ผู้อ่านออกแบบโครงสร้างที่สะท้อนทั้งสถาปัตยกรรมและโดเมนได้

16.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 17
- ไปสู่เล่ม 4 (Clean Architecture)

บทที่ 17: More than Concentric Layers: Hexagonal, Onion, Clean, CQRS

17.1 เรื่อง

การเชื่อม DDD กับสถาปัตยกรรมรูปแบบต่าง ๆ — Hexagonal, Onion, Clean, CQRS

17.2 วัตถุประสงค์

- อธิบายแต่ละรูปแบบ
- ทำให้เห็นความสัมพันธ์กับ DDD
- เตรียมพื้นฐานสำหรับการเลือกใช้

17.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- อธิบายแต่ละรูปแบบได้
- เห็นความสัมพันธ์กับ DDD
- เลือกใช้ให้เหมาะกับบริบท

17.4 หัวข้อหลัก

17.4.1 Hexagonal Architecture (Ports & Adapters)

- Core domain อยู่ศูนย์กลาง
- Port = interface ที่ domain เปิดให้
- Adapter = implementation ที่ infrastructure
- **ข้อดี:** testable, เปลี่ยนเทคโนโลยีง่าย
- **ข้อเสีย:** ต้องออกแบบ port ให้ดี

17.4.2 Onion Architecture

- ชั้น concentric โดย domain อยู่ศูนย์กลาง
- ชั้นนอกขึ้นกับชั้นใน
- คล้าย Hexagonal แต่มีชั้นชัดเจน

17.4.3 Clean Architecture

- คล้าย Onion แต่มีรายละเอียดชัดเจน
- Entities → Use Cases → Interface Adapters → Frameworks & Drivers
- Dependency Rule
- **รายละเอียดเพิ่ม:** เล่ม 4

17.4.4 CQRS

- แยก Command (write) และ Query (read)
- Write model เน้น consistency
- Read model เน้น performance
- **ข้อดี:** optimize แต่ละด้าน
- **ข้อเสีย:** ซับซ้อนขึ้น

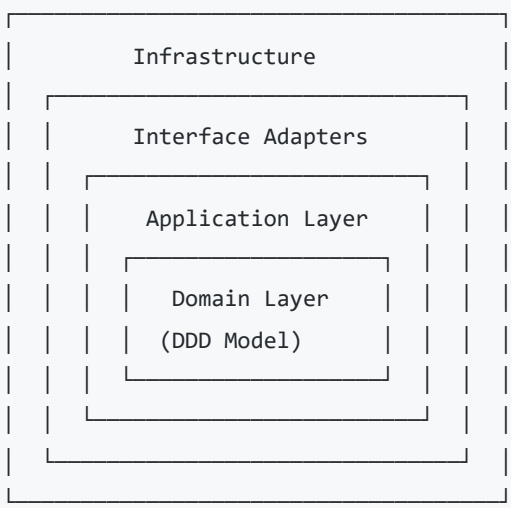
17.4.5 ความสัมพันธ์กับ DDD

แนวทาง	ใช้กับ DDD อย่างไร
Hexagonal	จัดโครงสร้างให้ domain อยู่ศูนย์กลาง
Onion	เช่นเดียวกัน แต่มีชั้นชัดเจน
Clean	เช่นเดียวกัน, มี dependency rule ชัด
CQRS	เสริมเมื่อ read/write ต่างกันมาก

17.4.6 การเลือกใช้

- **Hexagonal/Clean:** เหมาะกับระบบที่มี business logic ซับซ้อน
- **CQRS:** เหมาะกับ read/write ต่างกันมาก
- **Event Sourcing:** เหมาะกับ audit trail, complex domain

17.4.7 ตัวอย่างการประกอบกัน



17.5 ข้อดีของบท

- ทำให้เห็นภาพรวมของสถาปัตยกรรมที่ใช้กับ DDD

- เตรียมพื้นฐานสำหรับการเลือกใช้
- เชื่อมโยงกับเล่ม 4

17.6 ข้อเสีย/ข้อจำกัด

- เป็นภาพรวม ยังไม่ลงรายละเอียด
- ต้องอ่านเล่ม 4 สำหรับ Clean Architecture

17.7 ข้อควรระวัง

- อย่าใช้ทุกอย่างพร้อมกันโดยไม่จำเป็น
- อย่าเลือกตามกระแส
- อย่าละเลยความซับซ้อนที่เพิ่มขึ้น

17.8 ตัวอย่าง

ระบบ e-commerce:

- ใช้ **Hexagonal** สำหรับ Sales context
- ใช้ **CQRS** สำหรับ Product Catalog (query เยอะ)
- ใช้ **Event Sourcing** สำหรับ Order (audit trail)

17.9 การประยุกต์ใช้

- ใช้เลือกสถาปัตยกรรมให้เหมาะกับ context
- ใช้ใน architecture review
- ใช้ใน POC

17.10 สรุป/ผลลัพธ์

ผู้อ่านอธิบายและเลือกใช้รูปแบบต่าง ๆ ได้

17.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 18
- ไปสู่เล่ม 4 (Clean Architecture)

ส่วนที่ 5: การนำไปใช้และกรณีศึกษา

บทที่ 18: ตัวอย่างการออกแบบระบบด้วย DDD

18.1 เรื่อง

กรณีศึกษาการออกแบบระบบด้วย DDD ตั้งแต่ Strategic ถึง Architecture

18.2 วัตถุประสงค์

- ประยุกต์แนวคิดทั้งหมดกับกรณีศึกษา
- ทำให้เห็นภาพรวมการออกแบบ
- เตรียมผู้อ่านสำหรับการออกแบบระบบของตนเอง

18.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- ออกแบบระบบด้วย DDD ได้
- ใช้เครื่องมือจากบทก่อน ๆ
- วิเคราะห์ trade-offs

18.4 หัวข้อหลัก

18.4.1 บริบท

ระบบ e-commerce สำหรับ SME:

- ผู้ใช้: 1,000 คน/วัน
- ทีม: 15 คน แบ่งเป็น 3 ทีม
- ต้องการ deploy อิสระ
- Business logic ซับซ้อน (pricing, discount, promotion)

18.4.2 Strategic DDD

Subdomains:

- **Core:** Pricing & Promotion (สร้างความแตกต่าง)
- **Supporting:** Order Management, Inventory
- **Generic:** Payment, Notification, Authentication

Bounded Contexts:

- Sales Context
- Inventory Context
- Payment Context
- Notification Context
- Identity Context

Ubiquitous Language (Sales Context):

- Order, Cart, LineItem, Checkout, Promotion, Discount

Context Map:

```
Sales ──▶ Inventory (Customer/Supplier)
Sales ──▶ Payment (OHS)
Sales ──▶ Notification (Event)
```

18.4.3 Tactical DDD (Sales Context)

Aggregates:

- **Order Aggregate** — Order (root), OrderItem, Money
- **Cart Aggregate** — Cart (root), CartItem, Money
- **Promotion Aggregate** — Promotion (root), DiscountRule

Entities:

- Order, Cart, Promotion

Value Objects:

- Money, OrderId, CustomerId, Address, Discount

Domain Events:

- OrderPlaced, OrderConfirmed, OrderCancelled

Repositories:

- OrderRepository, CartRepository, PromotionRepository

Domain Services:

- PricingService (**คำนวณราคารวมกับ** promotion)

18.4.4 Architecture

โครงสร้าง:

```
sales/
├── domain/
│   ├── model/
│   │   ├── Order.java
│   │   ├── OrderItem.java
│   │   ├── Money.java
│   │   └── OrderId.java
│   ├── repository/
│   │   └── OrderRepository.java
```

```
│   ├── service/
│   │   └── PricingService.java
│   └── event/
│       └── OrderPlaced.java
├── application/
│   └── PlaceOrderUseCase.java
├── infrastructure/
│   └── JpaOrderRepository.java
└── api/
    └── OrderController.java
```

18.4.5 การทำงานของ PlaceOrder

1. **API:** รับ request
2. **Application:** PlaceOrderUseCase
3. **Domain:** Order.create(), PricingService.calculate()
4. **Repository:** save
5. **Event:** publish OrderPlaced

18.4.6 บทเรียน

- Strategic DDD ช่วยกำหนดขอบเขต
- Tactical DDD ช่วยออกแบบโมเดล
- Architecture ทำให้โมเดลอยู่ศูนย์กลาง
- Ubiquitous language สำคัญ
- Context mapping ช่วยจัดการ dependency

18.5 ข้อดีของบท

- ทำให้เห็นภาพรวมการออกแบบ
- เชื่อมแนวคิดทั้งหมดเข้าด้วยกัน
- เตรียมผู้อ่านสำหรับการออกแบบระบบของตนเอง

18.6 ข้อเสีย/ข้อจำกัด

- เป็นกรณีศึกษาเดียว
- ตัวอย่างอาจไม่ครอบคลุมทุกกรณี

18.7 ข้อควรระวัง

- อย่าคัดลอกโดยไม่ปรับบริบท
- อย่าละเลย trade-offs
- อย่าละเลยการวัดผล

18.8 ตัวอย่าง

Order flow:

1. User เพิ่มสินค้าใน cart
2. User checkout → PlaceOrderUseCase
3. Order ถูกสร้างด้วย PricingService
4. Order ถูก save
5. OrderPlaced event ถูก publish
6. Inventory Context รับ event และ reserve stock
7. Payment Context รับ event และ process payment

18.9 การประยุกต์ใช้

- ใช้เป็น template สำหรับระบบของตนเอง
- ใช้ใน workshop
- ใช้ใน architecture review

18.10 สรุป/ผลลัพธ์

ผู้อ่านออกแบบระบบด้วย DDD ได้ และเห็นภาพรวมการทำงาน

18.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่บทที่ 19
- ไปสู่เล่ม 4 (Clean Architecture)
- ไปสู่เล่ม 2 (Microservices)

บทที่ 19: การประยุกต์ใช้และสรุปเล่ม

19.1 เรื่อง

การนำเนื้อหาทั้งเล่มไปใช้ในองค์กร

19.2 วัตถุประสงค์

- สรุปสาระสำคัญของเล่ม
- แปลงความรู้เป็นแผนปฏิบัติ
- เชื่อมไปยังเล่มถัดไป

19.3 เป้าหมาย

เมื่อจบบท ผู้อ่านสามารถ:

- มีแผนประยุกต์ใช้ในองค์กร
- จัด workshop ตามแม่แบบในเล่ม
- รู้ว่าต้องไปอ่านเล่มใดต่อ

19.4 หัวข้อหลัก

19.4.1 Workshop: Domain Storytelling

- ใช้ภาคผนวก A
- เล่าเรื่อง业务流程
- ระบุ domain และ subdomain

19.4.2 Workshop: Bounded Context Canvas

- ใช้ภาคผนวก B
- ระบุ ubiquitous language
- กำหนดขอบเขต

19.4.3 Workshop: Context Mapping

- ใช้ภาคผนวก D
- ระบุความสัมพันธ์
- เลือกรูปแบบ

19.4.4 Workshop: Aggregate Design

- ใช้ภาคผนวก E
- ระบุ invariants
- ออกแบบ aggregate

19.4.5 สรุปสาระสำคัญ

- DDD มีสองระดับ: Strategic และ Tactical
- Strategic: domain, subdomain, bounded context, ubiquitous language, context mapping
- Tactical: entity, value object, aggregate, repository, domain event
- Architecture: Hexagonal, Onion, Clean, CQRS
- Ubiquitous language เป็นหัวใจ
- Context mapping ช่วยจัดการ dependency

19.4.6 เชื่อมไปเล่ม 4–7

- เล่ม 4: Clean Architecture (รายละเอียด)

- เล่ม 5: ความเสี่ยง
- เล่ม 6: POC
- เล่ม 7: การบริหารโครงการ

19.5 ข้อดีของบท

- ปิดเล่มด้วยการลงมือทำ
- ทำให้ความรู้แปลงเป็นแผนปฏิบัติ
- เชื่อมโยงกับเล่มถัดไปอย่างชัดเจน

19.6 ข้อเสีย/ข้อจำกัด

- ต้องปรับตามบริบทองค์กร
- ต้องมีผู้นำ workshop

19.7 ข้อควรระวัง

- อย่านำไปใช้แบบคัดลอกโดยไม่ปรับบริบท
- อย่าทำ workshop ครั้งเดียวแล้วจบ
- อย่าละเลยการติดตามผล

19.8 ตัวอย่าง

แผนนำไปใช้ใน 90 วัน:

- **เดือน 1:** Domain Storytelling + Bounded Context Canvas
- **เดือน 2:** Context Mapping + เลือก context ที่จะเริ่ม
- **เดือน 3:** Tactical DDD สำหรับ context แรก

19.9 การประยุกต์ใช้

- ใช้เป็นแผนดำเนินงานหลังอ่านจบ
- ใช้เป็น checklist ของทีม
- ใช้เป็นพื้นฐานสำหรับเล่มถัดไป

19.10 สรุป/ผลลัพธ์

ผู้อ่านมีแผนประยุกต์ใช้ในองค์กร และรู้เส้นทางการอ่านต่อ

19.11 เชื่อมโยงกับเล่มอื่น

- ไปสู่เล่ม 4–7 ทั้งหมด

ภาคผนวก

ภาคผนวก A: Domain Storytelling Canvas

หัวข้อ	คำอธิบาย	ตัวอย่าง
Actor	ผู้ที่กระทำ	Customer, Admin
Work Object	สิ่งที่ถูกกระทำ	Order, Product
Activity	สิ่งที่ทำ	Place order, Ship
Sequence	ลำดับ	1. Customer places order 2. System confirms
Domain	โดเมน	E-commerce

ภาคผนวก B: Bounded Context Canvas

หัวข้อ	คำอธิบาย	ตัวอย่าง
Name	ชื่อ context	Sales
Purpose	วัตถุประสงค์	จัดการคำสั่งซื้อ
Ubiquitous Language	คำศัพท์	Order, Cart, Checkout
Business Rules	กฎธุรกิจ	Order ต้องมี item อย่างน้อย 1
Aggregates	Aggregate หลัก	Order, Cart
Domain Events	Events	OrderPlaced
Dependencies	Context ที่พึ่งพา	Inventory, Payment
Team	ทีม	Sales Team

ภาคผนวก C: Ubiquitous Language Glossary Template

คำ	ความหมาย	ตัวอย่างการใช้	หมายเหตุ
Order	คำสั่งซื้อของลูกค้า	“Create a new order”	ต่างจาก Cart
Cart	ตะกร้าสินค้าก่อนสั่งซื้อ	“Add to cart”	ยังไม่ใช่ Order
LineItem	รายการสินค้าใน Order	“Order has 3 line items”	
Checkout	กระบวนการยืนยัน Order	“Proceed to checkout”	

ภาคผนวก D: Context Map Template

From	To	Pattern	ทิศทาง	หมายเหตุ
Sales	Inventory	Customer/Supplier	Sales → Inventory	Sales เป็น upstream
Sales	Payment	Open Host Service	Sales → Payment	Payment เปิด API
Shipping	Payment	Anticorruption Layer	Shipping → Payment	Shipping แปลงโมเดล

ภาคผนวก E: Aggregate Design Checklist

- [ ] ระบุ invariants ที่ต้องรักษา
- [ ] กำหนด boundary ที่เล็กที่สุด
- [ ] ระบุ aggregate root
- [ ] ออกแบบ behavior ที่รักษา invariants
- [ ] หลีกเลี่ยง aggregate ที่ใหญ่เกินไป
- [ ] อ้างถึง aggregate อื่นด้วย identity
- [ ] หนึ่ง transaction ต่อหนึ่ง aggregate
- [ ] ใช้ eventual consistency ระหว่าง aggregate

ภาคผนวก F: Domain Event Catalog Template

Event	เมื่อเกิด	ข้อมูล	ผู้สนใจ
OrderPlaced	เมื่อ Order ถูกสร้าง	orderId, customerId, total	Inventory, Payment, Notification
OrderConfirmed	เมื่อ Order ถูกยืนยัน	orderId, confirmedAt	Shipping, Notification
PaymentReceived	เมื่อได้รับการชำระเงิน	orderId, paymentId, amount	Sales, Notification

---

ภาคผนวก G: Glossary เฉพาะเล่ม 3

คำ	ความหมายโดยย่อ
Domain	ขอบเขตธุรกิจที่ระบบรองรับ
Subdomain	ส่วนย่อยของ domain
Core Domain	หัวใจของธุรกิจ
Supporting Subdomain	สนับสนุน core domain
Generic Subdomain	งานทั่วไปที่ซื้อ/ใช้ได้
Bounded Context	ขอบเขตของโมเดลและภาษา
Ubiquitous Language	ภาษากลางที่ทุกคนใช้ร่วมกัน
Context Map	แผนที่ความสัมพันธ์ระหว่าง context
Anticorruption Layer	ชั้นแปลงระหว่าง context
Entity	วัตถุที่มี identity
Value Object	วัตถุที่ไม่มี identity
Aggregate	กลุ่มของ entity/value object ที่มี boundary
Aggregate Root	Entity หลักของ aggregate
Repository	Abstraction สำหรับเข้าถึง aggregate

คำ	ความหมายโดยย่อ
Domain Event	เหตุการณ์ทางธุรกิจ
Domain Service	Operation ที่ไม่เป็นธรรมชาติของ entity
Factory	Abstraction สำหรับสร้าง object
Hexagonal Architecture	สถาปัตยกรรมที่แยก core ออกจาก infrastructure
Onion Architecture	สถาปัตยกรรมชั้น concentric
Clean Architecture	สถาปัตยกรรมที่แยกชั้นตาม dependency rule
CQRS	แยก read model กับ write model
Event Sourcing	เก็บ events เป็น source of truth

เกณฑ์ประเมินความสำเร็จของเล่ม

เกณฑ์	ตัวชี้วัด	น้ำหนัก
ความเข้าใจ	อธิบาย Strategic/Tactical DDD ได้	20%
Strategic	มี Bounded Context Canvas และ Context Map	25%
Tactical	มี Aggregate Design และ Domain Event Catalog	25%
Architecture	มีโครงสร้างโค้ดที่สะท้อนโดเมน	20%
การส่งต่อ	เข้าใจว่าบทใดเชื่อมไปเล่ม 4	10%

สรุปเล่ม 3

เล่ม 3 ทำหน้าที่เป็นคู่มือสำหรับการออกแบบซอฟต์แวร์ที่อิงธุรกิจเป็นศูนย์กลางด้วย DDD ผู้อ่านจะได้:

1. **ความเข้าใจ DDD** — Strategic และ Tactical
2. **Strategic DDD** — domain, subdomain, bounded context, ubiquitous language, context mapping
3. **Tactical DDD** — entity, value object, aggregate, repository, domain event, domain service, factory

4. **Architecture** — Hexagonal, Onion, Clean, CQRS
5. **โครงสร้างโค้ด** — ที่สะท้อนทั้งสถาปัตยกรรมและโดเมน
6. **กรณีศึกษา** — ตัวอย่างการออกแบบระบบจริง

เมื่ออ่านจบ ผู้อ่านควรมีความพร้อมในการออกแบบระบบที่ซับซ้อนด้วย DDD และรู้ว่าเมื่อใดควรใช้เครื่องมือแต่ละอย่าง
